

ELE 216 Laboratoire 4 : Modules et types opaques partie 2

Pour chacun des problèmes suivants, écrivez un module complet : un fichier d'en-tête (.h) qui définit l'interface publique, un fichier source (.c) qui implémente le module avec un type opaque, un fichier de tests unitaires utilisant Unity, et un Makefile. Documentez vos fichiers d'en-tête à mesure en format Doxygen.

Vous devez réaliser ce laboratoire seul, pendant la période. Il vous est permis de discuter avec vos collègues, consulter les chargés de laboratoires et l'internet.

Il est permis de réutiliser certaines fonctions réalisées dans les numéros précédents pour les numéros suivants.

Vous devez faire valider votre travail avant de quitter. On vous demandera d'expliquer clairement le fonctionnement de certains éléments de votre implémentation.

Problème 1 : Compteur d'événements

Modélisez un compteur qui surveille un seuil d'alerte. Chaque incrémentation rapproche le compteur du seuil ; lorsque le seuil est atteint, le nombre de dépassements augmente et la valeur courante revient à zéro. Le seuil est fixé à la création et ne peut pas changer par la suite.

Vous pouvez le visualiser comme un compteur qui se remet à zéro automatiquement une fois le seuil atteint et qui compte le nombre de tours.

État interne :

- valeur courante (entier)
- seuil de dépassement (entier, immuable)
- nombre de dépassements depuis la création (entier)

Interface :

`compteur_t *compteur_creer(int seuil) ;` alloue et initialise. Retourne NULL si $\text{seuil} \leq 0$.

`void compteur_detruire(compteur_t *c) ;` libère la mémoire.

`void compteur_incrementer(compteur_t *c) ;` augmente la valeur de 1. Si la valeur atteint le seuil, le nombre de dépassements augmente et la valeur revient à 0.

`void compteur_reinitialiser(compteur_t *c) ;` remet la valeur courante à 0. Le nombre de dépassements n'est PAS affecté.

`int compteur_valeur(const compteur_t *c) ;` retourne la valeur courante.

`int compteur_seuil(const compteur_t *c);` retourne le seuil.

`int compteur_depassements(const compteur_t *c);` retourne le nombre de dépassements.

`void compteur_afficher(const compteur_t *c);` affiche tous les champs sur le terminal.

Tests unitaires :

- création avec seuil valide ; création avec seuil invalide retourne NULL
- série d'incrémentations sans atteindre le seuil
- atteinte exacte du seuil : un dépassement, valeur = 0
- plusieurs cycles consécutifs de dépassement
- la réinitialisation préserve le compteur de dépassements

Problème 2 : Servo-moteur

Modélisez un servo-moteur avec butées physiques et une vitesse angulaire constante. Les butées (angle minimal et maximal) ainsi que la vitesse sont des caractéristiques matérielles, fixées à la création.

État interne :

- angle courant en degrés (float)
- angle minimum en degrés (float, immuable)
- angle maximum en degrés (float, immuable)
- vitesse en degrés/seconde (float, immuable)

Interface :

`servo_t *servo_creer(float angle_min, float angle_max, float vitesse);` alloue et initialise. Retourne NULL si $\text{angle_min} \geq \text{angle_max}$ ou $\text{vitesse} \leq 0$. L'angle courant initial est égale à l'angle minimum.

`void servo_detruire(servo_t *s);` libère la mémoire.

`void servo_aller_a(servo_t *s, float angle);` déplace le servo. Saturation aux butées si l'angle est inférieur au minimum ou supérieur au maximum.

`float servo_temps_pour_atteindre(const servo_t *s, float angle);` retourne le temps en secondes pour atteindre cet angle depuis la position courante (sans modifier l'état). Retourne une valeur négative si l'angle est en dehors des butées.

`float servo_angle(const servo_t *s);` retourne l'angle courant.

`float servo_angle_min(const servo_t *s);` retourne la butée minimale.

`float servo_angle_max(const servo_t *s);` retourne la butée maximale.

`float servo_vitesse(const servo_t *s);` retourne la vitesse.

`void servo_afficher(const servo_t *s);` affiche tous les champs.

Tests unitaires à inclure :

- création valide / création invalide ($\text{min} \geq \text{max}$, $\text{vitesse} \leq 0$)
- déplacement à l'intérieur de la plage normale
- consigne au-dessus du max → saturation à `angle_max`
- consigne sous le min → saturation à `angle_min`
- calcul du temps de déplacement pour un angle connu

Mini app : Moniteur de batterie

Un moniteur de batterie surveille l'état d'une pile au fil du temps. Il conserve un historique des pourcentages mesurés, applique un filtre passe-bas pour atténuer le bruit, et déclenche une alerte lorsque la charge filtrée descend sous un seuil.

Vous devez réutiliser les modules pile, buffer et filtre du laboratoire 3.

Structure à définir

Le moniteur encapsule les sous-modules suivants :

- une pile (le composant surveillé)
- un buffer FIFO contenant l'historique des pourcentages mesurés (entiers)
- un filtre moyenne mobile sur les pourcentages mesurés
- un seuil d'alerte en pourcentage (float, immuable)

Interface

`moniteur_t *moniteur_creer(float capacite, float tension, int taille_historique, int taille_filtre, float seuil_alerte);` alloue le moniteur ET tous ses sous-modules. Retourne NULL si l'un des sous-modules échoue à s'allouer (et libère ce qui avait déjà été alloué avant l'échec).

`void moniteur_detruire(moniteur_t *m);` libère le moniteur ET tous ses sous-modules.

`void moniteur_charger(moniteur_t *m, float mAh) ;` applique une charge à la pile interne. Ne modifie pas l'historique ni le filtre.

`void moniteur_decharger(moniteur_t *m, float mAh) ;` applique une décharge à la pile interne. Ne modifie pas l'historique ni le filtre.

`void moniteur_enregistrer(moniteur_t *m) ;` lit le pourcentage courant de la pile et l'ajoute à la fois à l'historique et au filtre.

`float moniteur_charge_brute(const moniteur_t *m) ;` retourne le pourcentage instantané de la pile (lecture directe).

`float moniteur_charge_lissee(const moniteur_t *m) ;` retourne le pourcentage filtré.

`int moniteur_alerte(const moniteur_t *m) ;` retourne 1 si la charge lissée est inférieure au seuil d'alerte, 0 sinon.

`void moniteur_afficher(const moniteur_t *m) ;` affiche l'état de la pile, le contenu de l'historique, la charge lissée et l'état de l'alerte.

Tests unitaires

- création du moniteur (valide/invalidé)
- séquence de charges et décharges, charge brute lue correctement
- la charge lissée converge vers la charge brute après suffisamment d'enregistrements stables
- l'alerte se déclenche au franchissement du seuil
- l'alerte se désactive après une recharge suffisante

Programme principal (tests manuels)

Le main orchestre une simulation :

- crée un moniteur (pile 5400 mAh, 11.7 V, historique 10 lectures, filtre fenêtre 5, seuil d'alerte 20 %)
- charge la pile à pleine capacité, enregistre une lecture
- effectue une vingtaine de cycles de décharge progressive, en enregistrant à chaque étape et en affichant, à chaque enregistrement, la charge brute, la charge lissée et l'état d'alerte (faites une fonction!)
- affiche l'état complet du moniteur en fin de simulation
- libère toute la mémoire